



Instituto Politécnico Nacional

Escuela Superior de Computo

Sistemas en Chip

Maestro: Perez Perez Jose Juan

Practica 2:

Decodificador

Integrantes:

- Ayala Fuentes Sunem Gizeht
- Guevara Badillo Areli Alejandra
- León Pérez May Carolina

Equipo: 1

Grupo 6CV1

Introducción

“Decodificador del ATmega8535 en ensamblador”

El siguiente programa en lenguaje ensamblador para el microcontrolador ATmega8535 implementa un decodificador para display de siete segmentos. Su función principal es convertir una entrada binaria recibida por el puerto B (PINB) en el número correspondiente mostrado en el display conectado al puerto A (PORTA).

Para lograrlo, el código utiliza una tabla de consulta (Lookup Table) almacenada en la memoria de programa, donde cada posición contiene el patrón binario que enciende los segmentos adecuados del display para representar los dígitos del 0 al F en formato hexadecimal. El programa lee continuamente el valor del puerto de entrada, accede al valor correspondiente en la tabla mediante los registros del puntero Z (ZH y ZL) y envía ese patrón a la salida.

De esta manera, se demuestra el uso de conceptos vistos en clase y la programación de microcontroladores.

Planteamiento del ejercicio:

Diseñar e implementar un decodificador utilizando el microcontrolador ATmega8535 programado en lenguaje ensamblador, cuya función principal sea recibir una entrada en formato binario y mostrar su valor equivalente en formato hexadecimal a través de un display de 7 segmentos.

Material y equipo usado:

- ❖ Jumpers
- ❖ ATmega8535
- ❖ Un display de 7 segmentos de cátodo común.
- ❖ 1 dipswitch
- ❖ Fuente de voltaje a 5V
- ❖ 2 pinzas banana caiman

Solución del problema:

; Descripción: Lee una entrada en binario desde el puerto B y muestra su valor hexadecimal en un display de 7 segmentos conectado al puerto A.

.include "m8535def.inc" ; Incluye definiciones específicas del microcontrolador ATmega8535

.def aux = R16 ; Define el registro R16 con el alias 'aux' (usado como auxiliar)

; Configuración De Puertos

ser aux ; Carga 0xFF (todos los bits en 1) en el registro 'aux'

out DDRA, aux ; Configura el Puerto A como SALIDA (para el display)

out PORTB, aux ; Configura el Puerto B con resistencias pull-up (entradas activas en bajo)

; Bucle Principal

NVO:

ldi zh, high(tabla<<1) ; Carga en ZH la parte alta de la dirección de la tabla

ldi zl, low(tabla<<1) ; Carga en ZL la parte baja de la dirección de la tabla

; -> El registro Z (ZH:ZL) apunta al inicio de la tabla

in aux, PINB ; Lee el valor del Puerto B (entrada en binario)

add zl, aux ; Suma el valor leído al puntero de la tabla (Z apunta al dato correspondiente)

lpm aux, z ; Carga desde la memoria de programa el valor de la tabla (según la entrada)

out PORTA, aux ; Envía el valor leído al Puerto A (para mostrarlo en el display)

rjmp NVO ; Regresa al inicio del bucle (ejecución continua)

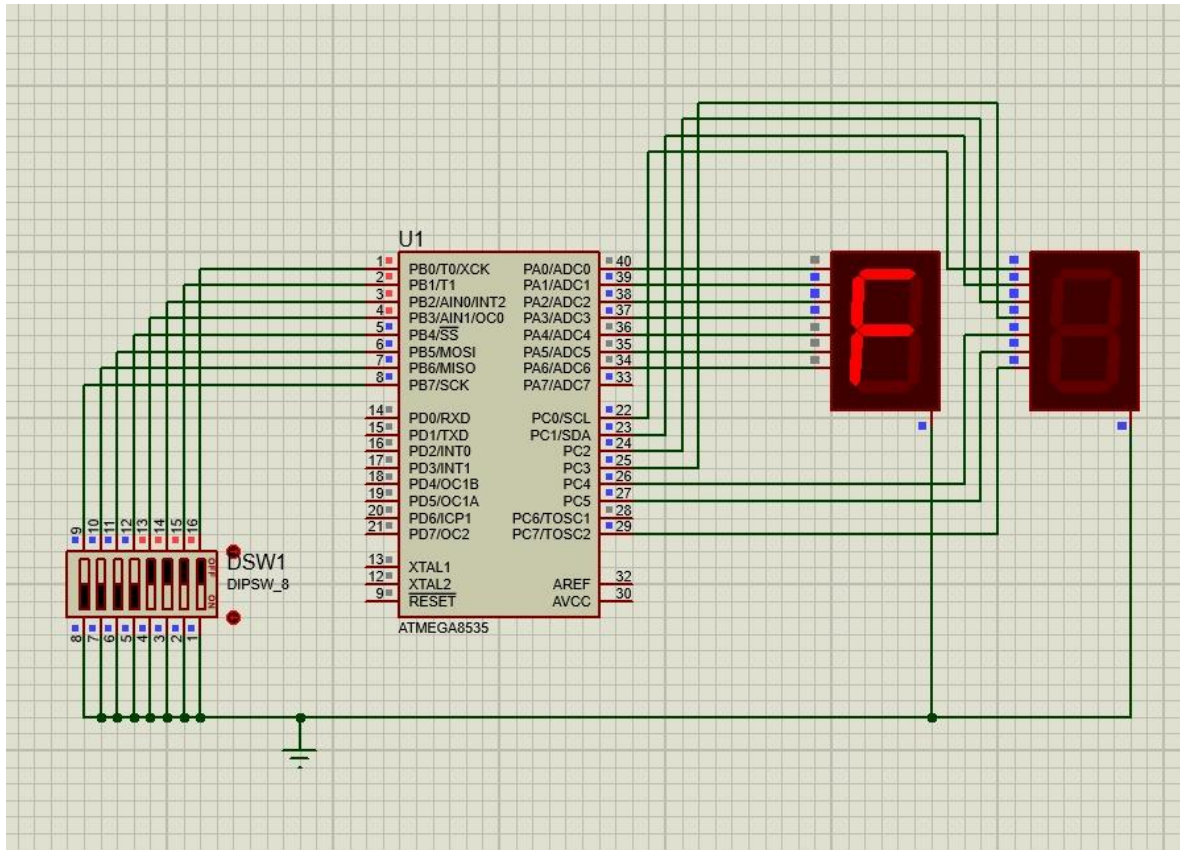
; Tabla de conversión, contiene los códigos de activación del display de 7 segmentos para representar los valores 0-F en formato hexadecimal.

tabla:

.db \$3f, \$06, \$5b, \$4f, \$66, \$6d, \$7d, \$27, \$7f, \$6f, \$77, \$7c, \$39,
\$5e, \$79, \$71

; 0 1 2 3 4 5 6 7 8 9 A b C d E F

Diagrama del circuito:



Conclusiones:

Ayala Fuentes Sunem Gizeht:

En esta práctica se implementamos con éxito una tabla de consulta (Lookup Table) para el control de un display de siete segmentos, lo que nos permitió optimizar la conversión de datos de forma eficiente y estructurada. Con este enfoque reemplazamos el uso de múltiples comparaciones condicionales, simplificando el código y mejorando su rendimiento. La utilización de la tabla de consulta demostró ser una técnica eficaz para reducir la carga de procesamiento en tiempo de ejecución, ya que las salidas requeridas se encuentran previamente definidas en la memoria del programa, evidenciando un método de programación más ordenado y rápido.

Guevara Badillo Areli Alejandra:

La conclusión principal de esta práctica es la implementación de una tabla de consulta (Lookup Table) para controlar un display de 7 segmentos. Esta técnica no solo cumplió el objetivo funcional, sino que demostró ser una solución altamente eficiente para la conversión de datos. En lugar de recurrir a un largo y complejo bloque de código con múltiples comparaciones, se optó por un enfoque que optimiza tanto el rendimiento como la claridad del programa. El método de la tabla de consulta evita el procesamiento lógico en tiempo de ejecución, ya que todas las respuestas posibles ya están precalculadas y almacenadas en la memoria del programa.

Leon Perez May Carolina:

La práctica en si no es muy complicada, ya que es un simple decodificador de binario a 7 segmentos. Gracias a la simpleza de la práctica y la frecuencia con la que se realiza en otras materias como Fundamentos de Diseño Digital y Diseño de Sistemas Digitales, es más fácil comprender la realización de esta por medio de tablas. Las tablas nos permiten buscar el valor y coincidirlo directamente con la salida que corresponde en vez que usar una función booleana como se ha hecho o usando circuitos integrados. Esta es una buena optimización ya que permite realizarlo sin tantos cálculos.

Referencias:

- ❖ ATmega8535. (s/f). Microchip.com. Recuperado el 31 de octubre de 2025, de <https://www.microchip.com/en-us/product/atmega8535>
- ❖ Microprocessor Design/Instruction Decoder. (s/f). Wikibooks.org. Recuperado el 31 de octubre de 2025, de https://en.wikibooks.org/wiki/Microprocessor_Design/Instruction_Decoder
- ❖ Temas vistos en clase